

Algorithmen und Datenstrukturen

Übung 2: Grundlagen

Robert

2026-05-12

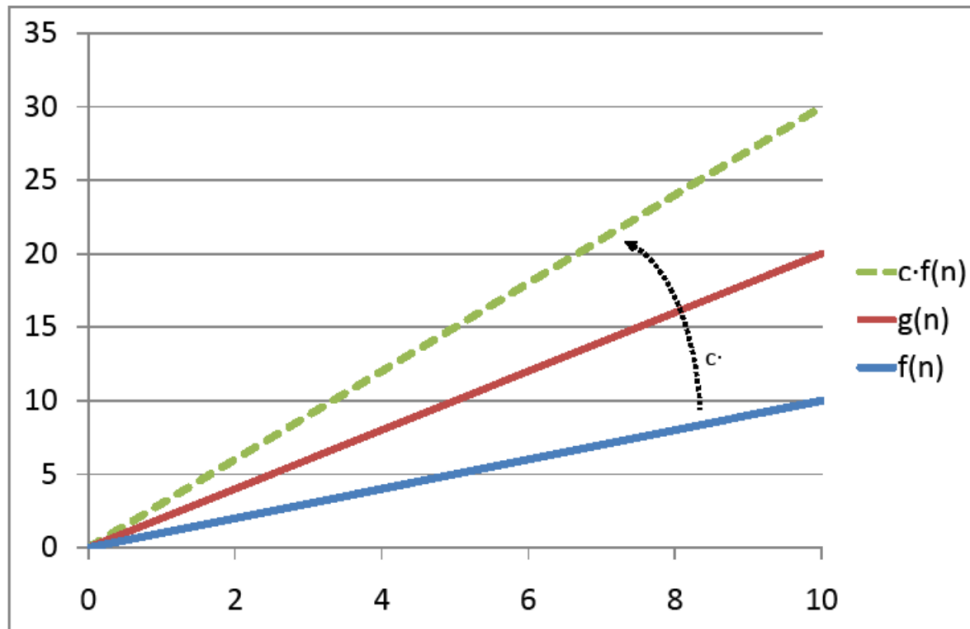
Wiederholung Laufzeitanalyse

Laufzeit

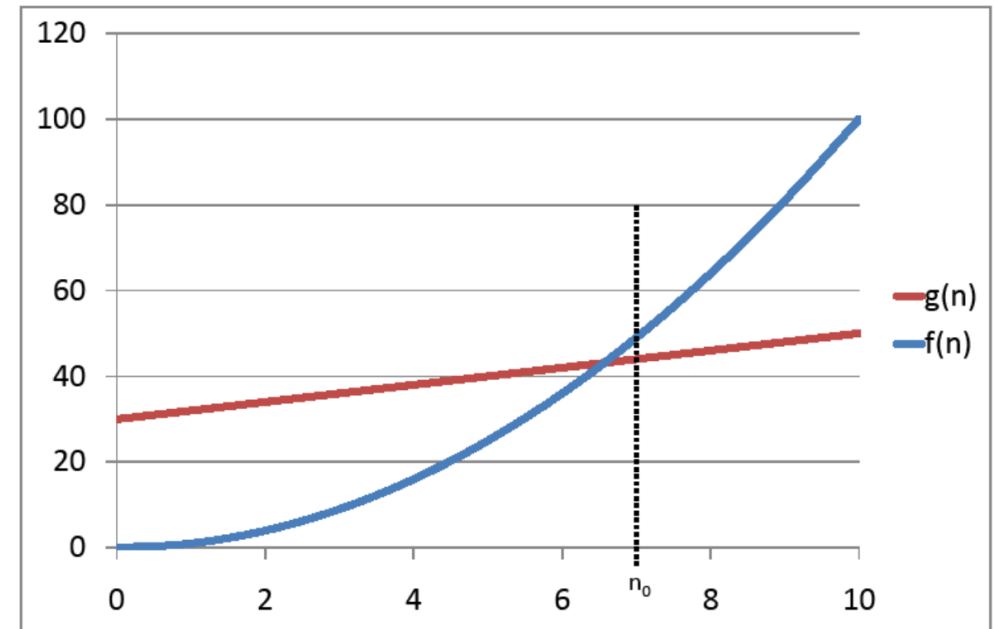
- Um die Laufzeit eines Algorithmus darzustellen wird die O-Notation verwendet $O(f)$ (wobei f eine Funktion ist).
- verschiedene Algorithmen haben verschiedene Laufzeiten, die in aufsteigende Komplexitätsklassen eingeteilt werden können:

$$O(\log(n)), O(n), O(n \cdot \log(n)), O(n^2), O(n^3)$$

- die exakte Laufzeit eines Algorithmus wird nicht angezeigt, sondern nur die Komplexitätsklasse, die wie eine obere Schranke ist
 - das heißt, die Laufzeit des Algorithmus überschreitet nie die Funktion der Komplexitätsklasse



- die Laufzeit $O(3n)$ gehört zur Komplexitätsklasse $O(n)$, da die Laufzeit mit einer Konstante c überschritten werden kann



- ein Algorithmus, der eine Laufzeit von $O(n)$ hat, kann ab n_0 die Komplexitätsklasse $O(n^2)$ nicht mehr überschreiten

Aufgabe 1

1 b)

```
1 public static boolean f1(String str1, String str2) {
2     char[] array1 = new char[Character.MAX_VALUE];
3     char[] array2 = new char[Character.MAX_VALUE];
4     for(char c : str1.toCharArray()){
5         array1[c]++;
6     }
7     for(char c : str2.toCharArray()){
8         array2[c]++;
9     }
10    for(char c = 0; c < Character.MAX_VALUE; c++){
11        if(array1[c] != array2[c])
12            return false;
13    }
14    return true;
15 }
```

1. Konstante Laufzeit $O(1)$:

- Arrayzuweisung (Zeile 5, 8)
- Arrayvergleich (Zeile 11)
 - ist immer `MAX_VALUE` lang
- return

2. Lineare Laufzeit $O(n)$:

- beide for-Schleifen (abhängig von Stringlänge)

Formel: $O(1) + 2 \cdot O(n) = O(n)$

1 c)

```
1 public static double f2(double a, int n) {  
2     double x = 1.0;  
3     for (int i = 0; i < n; i++)  
4         x = 0.5 * (x + a / x);  
5     return x;  
6 }
```

1. Konstante Laufzeit $O(1)$:

- Wertdefiniertung (Zeile 2)
- return (Zeile 5)

2. Lineare Laufzeit $O(n)$:

- for-Schleife (Zeile 3)

Formel: $O(1) + O(n) = O(n)$

1 d)

```
1 public static double f3(int n) {  
2     double sum = 0.0;  
3     for (int i = 0; i < n; i++) {  
4         double x = Math.random();  
5         double y = Math.random();  
6         if(Math.sqrt(x*x+y*y) <= 1.0)  
7             sum++;  
8     }  
9     return 4.0*sum/n;  
10 }
```

1. Konstante Laufzeit $O(1)$:

- Wertdefinierung (Zeile 2)
- return (Zeile 9)

2. Lineare Laufzeit $O(n)$:

- for-Schleife (Zeile 3)

Formel: $O(1) + O(n) = O(n)$

1 aa)

```
1 find(value, t):
2     t_current = t.root
3     while (t_current is not leaf):
4         if value == t_current.value:
5             return True
6         else if value > t_current.value:
7             t_current = t_current.right
8         else:
9             t_current = t_current.left
10    return value == t_current.value
```

1. Konstante Laufzeit $O(1)$:

- `t_current` (Zeile 2, 7, 9)
- `return` (Zeile 5, 10)

3. andere Laufzeiten:

- Suche in Binärbäumen hat $O(\log_2(n))$

Formel: $O(1) + O(\log_2(n)) = O(\log_2(n))$

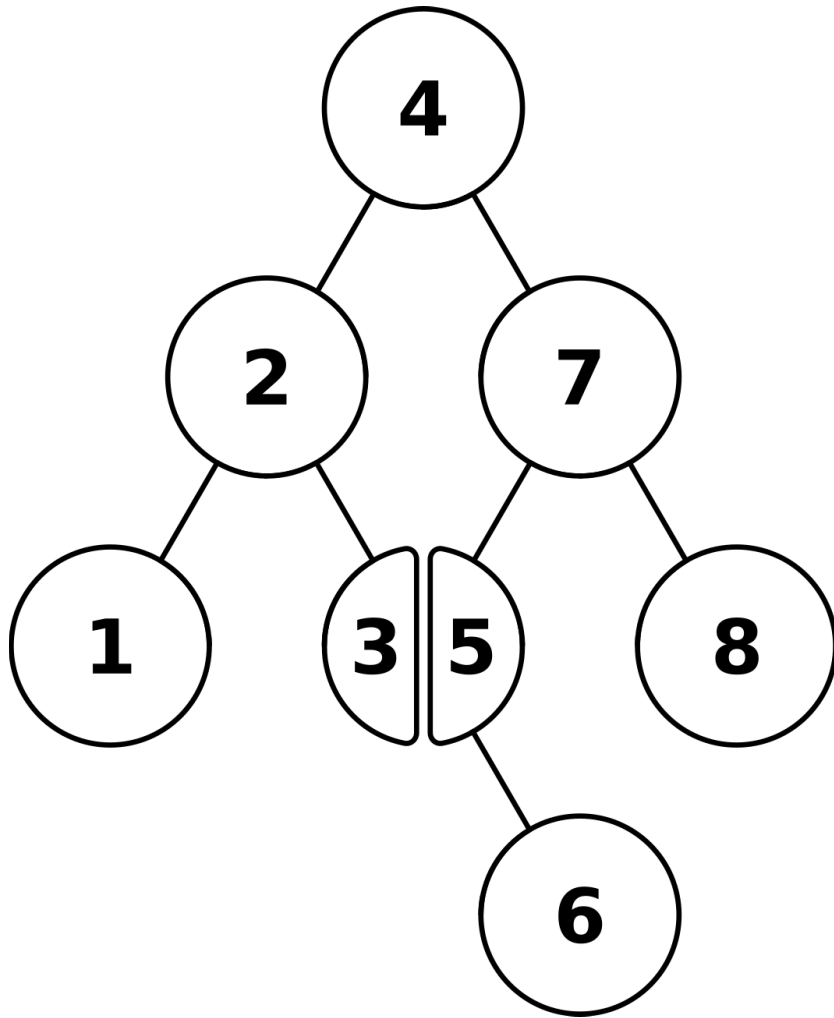
Suche in Binärbäumen

Warum dauert die Suche in einem balancierten Binärbaum mit n Elementen (Knoten) $\log_2(n)$?

- Bei einer Höhe h kann ein Binärbaum maximal $2^h - 1$ Elemente haben
- Daraus lässt sich folgende Formel ableiten: $n = 2^h - 1$
- Diese Formel lässt sich umformen zu:

$$h \approx \log_2(n)$$

Suche in Binärbäumen



- Die maximale Anzahl an Vergleichen, die in diesem Binärbaum durchgeführt werden müssen entspricht der Höhe, die proportional zu $\log_2(n)$ ist
- Das heißt $\log_2(n)$ ist die Obergrenze an Schritten und somit die Komplexitätsklasse

1 bb)

```
1 int compare (char[] a, char[] b) {  
2     int count = 0;  
3     for (char c : a) {  
4         for (char d : b) {  
5             if (c == d) count++;  
6         }  
7     }  
8     return count;  
9 }
```

1. Konstante Laufzeit $O(1)$:

- Wertdefinierung (Zeile 2)
- return (Zeile 8)

3. andere Laufzeiten $O(n^2)$:

- geschachtelte for-each (Zeile 3-4)

Formel: $O(1) + O(n^2) = O(n^2)$

Aufgabe 2

2 a)

$$O(n) \subseteq O(n^2)$$

- ist richtig, weil n^2 schneller anwächst als n

Beweis:

$$\lim_{n \rightarrow \infty} \frac{n}{n^2} = 0$$

2 b)

$$\log_{10}(x) \notin O(\log_2(x))$$

- ist falsch, alle logarithmischen Laufzeiten gehören zu gleichen Komplexitätsklasse

$$O(\log_5(n)) = O(\ln(n)) = O(\log_2(n))$$

Beweis: Basiswechsel (Vorlesung 2, Seite 16)

$$\log_a n = (\log_a b) \cdot \log_b n = \text{const} \cdot \log_b n$$

2 c)

$$O(n^2) \subseteq O(n \cdot \log_2(n))$$

- falsch, weil n^2 schneller anwächst als $n \cdot \log_2(n)$

Beweis:

$$\lim_{n \rightarrow \infty} \frac{n^2}{n \cdot \log(n)} \rightarrow \infty$$

2 d)

$$2000 \cdot g(n) \in O(g(n))$$

- richtig, weil konstante Faktoren (2000) vernachlässigt werden können

2 e)

$$g(n) \cdot n^2 \in O(n^2)$$

- ist nur richtig, wenn $g(n)$ eine konstante Laufzeit hat, also zu $O(1)$ gehört